섹션 C – 스택과 큐 [이 섹션에서 1문항 선택]

안내: 문제용 프로그램 템플릿은 APAS 시스템에서 제공됩니다. 반드시
해당 템플릿을 사용하여 함수를 구현하세요.

1. (createQueueFromLinkedList) createQueueFromLinkedList() C 함수를 작성하시오.

연결 리스트에 저장된 모든 정수를 순서대로 enqueue 하여
연결 리스트 기반 큐를 생성해야 한다. 첫 번째 노드부터 차례대로 enqueue 한다.
큐가 비어 있지 않다면 시작할 때 먼저 큐를 비워야 한다.

함수 원형은 다음과 같다:

```
void createQueueFromLinkedList(LinkedList *ll , Queue *q);
```

예시 입력/출력 세션은 다음과 같다. 현재 연결 리스트가 1,2,3,4,5 인 경우이다:

결과 연결 리스트는: 1 2 3 4 5

```
메뉴를 선택하세요(1/2/3/0): 2
결과 큐는: 1 2 3 4 5
```

```
}
    temp = temp->next;
    //do what you want//
while(temp != NULL){
ListNode *temp = ll->head;
```

2. (createStackFromLinkedList) createStackFromLinkedList() C 함수를 작성하시오.

연결 리스트에 저장된 모든 정수를 차례대로 push 하여
연결 리스트 기반 스택을 생성해야 한다. 첫 번째 노드부터 차례대로 push 한다.
스택이 비어 있지 않다면 시작할 때 먼저 스택을 비워야 한다.

함수 원형은 다음과 같다:

```
void createStackFromLinkedList(LinkedList *ll , Stack *stack);
```

예시 입력/출력 세션은 다음과 같다. 현재 연결 리스트가 1,3,5,6,7 인 경우이다:

결과 연결 리스트는: 1 3 5 6 7

```
메뉴를 선택하세요(1/2/3/0): 2
결과 스택은: 7 6 5 3 1
```

```
}
    temp = temp->next;
    //do what you want//
while(temp != NULL){
ListNode *temp = ll->head;
```

3. (isStackPairwiseConsecutive) 스택의 수들이 쌍별로 연속인지 검사하는

C 함수 isStackPairwiseConsecutive()를 작성하시오.
이 함수는 스택의 숫자들이 쌍별 연속인지 확인해야 한다. 또한
정수를 추가하거나 제거할 때 push()와 pop()만 사용해야 한다.

함수 원형은 다음과 같다:

```
int isStackPairwiseConsecutive(Stack *s);
```

예시 테스트 케이스는 다음과 같다:

모든 검사를 통과하면 1을 반환할 수 있다.
그 외에는 모든 쌍을 검사하고, 하나라도 조건을 만족하지 않으면 0을 반환
스택 크기가 홀수이면 0을 반환한다.

예를 들어 스택이 (16,15,11,10,5,4) 라면:

```
스택은: 16 15 11 10 5 4
```

스택의 원소들은 쌍별 연속이다

예를 들어 스택이 (16,15,11,10,5,1) 라면:

```
스택은: 16 15 11 10 5 1
```

스택의 원소들은 쌍별 연속이 아니다

예를 들어 스택이 (16,15,11,10,5) 라면:

```
스택은: 16 15 11 10 5
```
스택의 원소들은 쌍별 연속이 아니다

4. (reverseQueue) 스택을 사용하여 큐를 뒤집는 C 함수 reverseQueue()를 작성하시오.

reverseQueue() 함수는 스택에 정수를 추가/삭제할 때 push()와 pop()만 사용해야 하며,
큐에 정수를 추가/삭제할 때 enqueue()와 dequeue()만 사용해야 한다.
또한 스택이 비어 있지 않다면 시작할 때 먼저 스택을 비워야 한다.

스택을 초기화하는 것을 잊지 말 것.

함수 원형은 다음과 같다:

```
void reverseQueue(Queue *q);
```

예를 들어 큐가 (1,2,3,4,5) 라면 결과 큐는 (5,4,3,2,1) 이다.

5. (recursiveReverseQueue) 정수 큐에 저장된 항목의 순서를 재귀적으로 뒤집는

C 함수 recursiveReverseQueue()를 작성하시오.

enqueue(q,temp);
recursiveReverseQueue(q);
temp = dequeue(q);
`if(q->ll.head == NULL) return;`
`int temp;`

함수 원형은 다음과 같다:

```
void recursiveReverseQueue(Queue *q);
```

예를 들어 큐가 (1,2,3,4,5) 라면 결과 큐는 (5,4,3,2,1) 이다.

6. (removeUntilStack) 선택한 값이 처음 나타날 때까지

정수 스택의 값을 모두 pop 하는 C 함수 removeUntilStack()를 작성하시오.

s->ll.size == 0 인 경우를 고려할 것.
for 문도 while 문처럼 break 할 수 있다.

함수 원형은 다음과 같다:

```
void removeUntilStack(Stack *s, int value);
```

맨 위 원소가 왼쪽에 보이도록 스택이 (1,2,3,4,5,6,7) 일 때,
removeUntilStack()를 value = 4로 호출하면 스택은 (4,5,6,7)이 된다.

맨 위 원소가 왼쪽에 보이도록 스택이 (10,20,15,25,5) 일 때,
removeUntilStack()를 value = 15로 호출하면 스택은 (15,25,5)가 된다.

7. (balanced) 문자 () [] {} 로 이루어진 식이 균형 잡혀 있는지 판별하는

C 함수 balanced()를 작성하시오. balanced() 함수의 원형은
다음과 같다:

함수 원형은 다음과 같다:

```
int balanced(char *expression);
```

예를 들어 다음 식들은 괄호의 순서와 개수가 일치하므로 균형이 맞다:

()
([])
{[]()[]}

마지막에 스택이 비었는지 확인한다.
}
`if }]) match with the peek of stack then p`
`else {`
if {[(push into stack
스택을 사용할 것.

예시 입력/출력 세션은 다음과 같다:

```
1: 문자열 입력:
```

2: ()[]{} 문자로 이루어진 식이
균형이 맞는지 확인:
0: 종료:

메뉴를 선택하세요(1/0): 1
공백 없이 식을 입력하여
균형 여부를 확인하세요: {[] () []}
{[] () []}
균형 잡힘!

메뉴를 선택하세요(1/0): 0

다음 식들은 균형이 맞지 않는다:
{{)]
[({{)])

예시 입력/출력 세션은 다음과 같다:
1: 문자열 입력:
2: ()[]{} 문자로 이루어진 식이
균형이 맞는지 확인:
0: 종료:

메뉴를 선택하세요(1/0): 1
공백 없이 식을 입력하여
균형 여부를 확인하세요: [({{)])
[({{)])
균형이 맞지 않음!

메뉴를 선택하세요(1/0): 0